REPOMIND: Reproducing 256K-context Repository-Scale Code Understanding on a Single AMD MI300X with FP8 KV Cache

Methodology, Empirical Benchmarks, and an AITER Attention Backend Regression on Qwen3-Coder-Next-FP8

Author: Sardor Razikov [1]
Affiliation: [1] Independent ML Engineer, Tashkent, Uzbekistan
Contact: razikovsardor1@gmail.com
Code: https://github.com/SRKRZ23/repomind (MIT License)
Demo: https://huggingface.co/spaces/ZeroR3/repomind
Submission context: AMD Developer Hackathon 2026 (lablab.ai), submitted 2026-05-06

Abstract

We present **REPOMIND**, an open-source repository-scale coding agent that operates on a single AMD MI300X (192 GB HBM3) accelerator at 256K context length using `Qwen/Qwen3-Coder-Next-FP8` (80B parameters, 3B active MoE, FP8 weights and KV cache). We report **62 measured data points** collected across 124 minutes of stress testing on AMD Developer Cloud infrastructure: throughput as a function of context length, concurrency scaling at 8K–256K context windows, long-context coherence via needle-in-a-haystack probes at the 200K position, and end-to-end repository question-answering on three real codebases including `pytorch/vision` (~1.3M tokens, 5× the 256K window). All 31 parallel

users succeed at every realistic context (8K–64K, 31/31), 144/144 outputs are clean on the default Triton attention backend, the needle probe passes at all three positions including 200K, and all 9 end-to-end repository questions are answered correctly.

We also report an **engineering honesty result**: AMD's `ROCM_AITER_FA` attention backend, advertised for higher throughput on MI300X, produces **2–4× higher aggregate throughput** when combined with FP8 KV cache, but degenerates outputs to repeating punctuation (`"!!!!!!!!"`-style) on **137 of 144 cells** in our concurrency matrix on this specific model + configuration. The default ROCm Triton backend remains production-safe; AITER stays research-quality on this configuration as of vLLM 0.17.1 / ROCm 7.2.0. We file this regression with reproducible scripts and per-cell evidence in the public repository.

Total session cost: **4.12*ofcompute* * *(1.99/hr × 2.07 hr across two sessions on AMD Developer Cloud). We argue that a memory-architecture moat exists — `Qwen3-Coder-Next-FP8` weights (77.29 GiB) + FP8 KV cache (94.58 GiB) + activations approximates ~143 GiB, which does not fit on an NVIDIA H100 80GB single-card by VRAM accounting but has headroom on MI300X's 192 GB — and that this moat enables a category of on-premises repository-scale coding assistance that hosted SaaS coding tools cannot legally serve to compliance-locked enterprises (banks, defense, healthcare, IP-sensitive product teams).

Keywords: AMD MI300X, ROCm 7.2, Qwen3-Coder-Next, FP8 KV cache, vLLM, repository-scale code understanding, 256K context, needle-in-haystack, AITER attention backend regression, open-source coding agents, on-premises LLM deployment, compliance-locked AI coding.

---

1. Introduction

1.1 Motivation

Hosted SaaS coding assistants — Cursor, GitHub Copilot Business, Claude Team — are unavailable to a meaningful fraction of professional software developers by policy. JPMorgan Chase publicly restricted internal employee use of ChatGPT in early 2023, as reported by the Wall Street

Journal [1]. Apple subsequently restricted internal employee use of ChatGPT and GitHub Copilot, originally reported by the Wall Street Journal and widely re-reported by secondary financial press [2]. The U.S. Department of Defense, per published acquisition guidance, requires on-premises deployment for AI tooling that operates against sensitive code or data. Banks regulated under U.S. Federal Reserve SR 11-7 model risk management guidance, defense contractors under DFARS / CMMC handling controls, healthcare incumbents under HIPAA exposure, and IP-sensitive product teams in mobile and firmware all face structurally similar constraints: source code cannot leave the corporate VPC.

This is not a small population. Author's order-of-magnitude estimate (based on publicly reported headcount figures at the named institutions plus the top-tier U.S. banks, U.S. defense contractors, and major IP-sensitive product companies): **millions** of professional developers globally operate under such constraints. At Cursor's $40/seat/month tier (as published on Cursor's pricing page, https://cursor.com/pricing, accessed by the author) the addressable productivity market measured purely in SaaS substitution is on the order of tens of billions of dollars per year — and that market is *unreachable* by the SaaS coding incumbents themselves. Precise sizing is beyond the scope of this preprint; the order of magnitude is cited only to establish the motivation for an on-premises alternative.

The technical question is whether a single-host on-premises configuration can match the *capability surface* (long context, repository-scale code understanding, agentic tool use) of hosted coding agents. AMD's February 2026 technical article on Qwen3-Coder-Next claimed exactly this configuration is supported on MI300X with day-0 ROCm 7 [3]. REPOMIND is the open-source empirical proof.

1.2 Contribution

This work makes four contributions:

1. **A reproducible benchmark suite** (62 data points across 124 minutes of MI300X time) characterizing throughput, concurrency, long-context coherence, and end-to-end repository Q&A on `Qwen3-Coder-Next-FP8` with 256K context and FP8 KV cache, on a single AMD MI300X.

2. **Empirical validation of the memory-architecture moat**: weights + KV cache + activations measured to occupy ~143 GiB peak, which does not fit on NVIDIA H100 80GB by VRAM accounting.
3. **An open AITER attention backend regression report** with per-cell reproduction scripts: `ROCM_AITER_FA` × FP8 KV cache on `Qwen3-Coder-Next-FP8` produces broken outputs on 137 of 144 cells in our matrix despite a 2–4× throughput improvement, suggesting an FP8 KV cache scaling factor interaction that warrants upstream investigation.
4. **A simple production-ready agent loop** (PLAN → CALL TOOL → OBSERVE → THINK → ANSWER, adapted from a prior math-olympiad reasoning pipeline [4]) wrapping five tools (`read_file` , `grep_codebase` , `execute_code` , `run_tests` , `git_log`) that demonstrates the configuration in agentic use, including end-to-end repository question-answering on three real codebases.

We do not contribute novel model architecture, novel attention algorithms, or novel quantization techniques. We contribute reproducible evidence that an existing, free, MIT-licensed open-source stack (vLLM 0.17.1 + ROCm 7.2.0 + `Qwen3-Coder-Next-FP8`) on commodity AMD cloud infrastructure ($1.99/hr per MI300X) achieves the capability surface advertised, and we document the one tuning path we explored that did *not* work safely so others do not waste cloud time discovering it.

---

2. Background and Related Work

2.1 The MI300X memory-architecture moat

AMD's MI300X accelerator, introduced in late 2023 and broadly available throughout 2024–2026, ships with 192 GB of HBM3 memory at 5.3 TB/s aggregate bandwidth. NVIDIA's H100 80GB SXM (the dominant deployed inference GPU through 2025) ships with 80 GB of HBM3 at 3.35 TB/s. NVIDIA's H200 (announced 2023, available 2024) increased HBM to 141 GB; B100/B200 (Blackwell, 2024–2025) further expanded memory. As of this writing the population of deployed inference GPUs in compliance-locked enterprises is heavily weighted toward H100 80GB SXM — and the memory delta between 80 GB and 192 GB is the operational constraint of interest, not the bandwidth.

A 80B-parameter MoE model with 3B active parameters and FP8 weights occupies approximately 77 GiB of VRAM for weights alone. A 256K-context KV cache at FP8 occupies approximately 94 GiB. Activations and PyTorch overhead add ~10–15 GiB. The sum (~ 143 GiB) does not fit on a single H100 80GB SXM by VRAM accounting; on H200 (141 GB) it is at the absolute edge; on MI300X (192 GB) it has roughly 49 GB of headroom. Multi-GPU configurations of course allow H100 to serve this model, but the relevant comparison for an on-premises *single-host single-GPU* deployment scenario — the scenario most compliance-locked enterprises actually want — is the single-card VRAM ceiling.

2.2 Qwen3-Coder-Next-FP8

`Qwen/Qwen3-Coder-Next-FP8` is the FP8-quantized release of Alibaba Cloud's Qwen3-Coder family [5]. The model uses an MoE architecture (80B total parameters, 3B active per forward pass) with a maximum-supported context window of 262,144 tokens (256K). AMD's February 2026 technical article [3] documented day-0 ROCm 7 support for the FP8 variant. vLLM 0.17.1 (the version used in this work; release notes referenced in our citation [6]) serves the model via its native MoE routing path.

2.3 vLLM, ROCm, and the Triton vs AITER backend choice

vLLM [6] is the de facto open-source serving framework for LLM inference, offering continuous batching, paged attention, FP8 KV cache, and (on ROCm) two attention backend implementations: the default Triton-based implementation and the optimized AITER (`ROCM_AITER_FA`) implementation maintained by the AMD AI Group. AITER is broadly documented as providing higher throughput on MI300X for many production workloads; production AMD blogs [7] benchmark single-precision and BF16 configurations primarily.

The combination of **FP8 weights × FP8 KV cache × AITER attention backend × Qwen3-Coder-Next-FP8** is, to the author's knowledge as of submission, not extensively benchmarked in publicly available material (this is a literature-search statement, not an absence-of-publication proof). Section 5 below reports the experimental result on this specific combination, with reproducible scripts in the public repository.

2.4 Related open-source coding agents

OpenDevin [8], Aider [9], and SWE-agent [10] are widely-used open-source repository-scale code agents. They focus on agent architecture, tool design, and prompt patterns — typically operating against hosted LLM APIs (OpenAI, Anthropic, hosted Llama). REPOMIND's contribution is not in this layer; we adapt a simpler 5-tool SC-TIR-style loop [4] and focus the empirical effort on demonstrating that the *single-GPU on-premises configuration* works end-to-end at 256K context. The agent loop is intentionally minimal so the underlying serving configuration is the variable being measured.

---

3. Methodology

3.1 Hardware and software configuration

All measurements were collected on AMD Developer Cloud (ATL1 region, DigitalOcean-backed infrastructure). Hardware: one MI300X x1 droplet ($1.99/hour). Software image: vLLM 0.17.1 + ROCm 7.2.0 Quick Start (one-click DigitalOcean image, no manual ROCm install). Model: `Qwen/Qwen3-Coder-Next-FP8`. Serving command:

```
vllm serve Qwen/Qwen3-Coder-Next-FP8 \
  --max-model-len 262144 \
  --kv-cache-dtype fp8 \
  --port 8000 \
  --host 0.0.0.0 \
  --gpu-memory-utilization 0.92
```

Phase 1 used the default Triton attention backend. Phase 2 substituted `--attention-backend ROCM_AITER_FA` to evaluate AITER. All other parameters identical between phases.

3.2 Benchmark suite (Phase 1)

The Phase 1 suite consists of five components, all scripted in the public repository at `benchmarks/runner/`:

1. **Throughput per context** (`bench_throughput.json`): single-user TTFT (time-to-first-token) and decode-bound throughput across 8K, 32K, 64K, 128K, 256K context lengths. Streaming and non-streaming variants.
2. **Concurrency stress** (`bench_concurrency.json`): identical-prompt N-parallel sweeps at 32K, 128K, 256K with N ∈ {1, 8, 16, 31}. Twelve cells. p95 latency and aggregate throughput reported.
3. **Long-context coherence** (`bench_long_context.json`): a needle-in-a-haystack probe at three positions (early ~99K, middle ~199K, late ~99K). The needle is a unique function name (`calc_repomind_token_budget_v7`) plus a magic numerical constant (`4242`); pass = both substrings recovered.
4. **End-to-end repository Q&A** (`bench_e2e.json`): three repositories (REPOMIND itself, `pallets/flask`, `pytorch/vision`) × three questions each = nine cells. Pass = qualitative correctness verified by author (citations to actual file paths required).
5. **Cost economics** (`bench_cost.json`): $/M tokens, break-even calculations vs Cursor Business pricing at typical team sizes.

Total Phase 1 wall clock: 97 minutes. Total Phase 1 cost: ~$3.22.

3.3 Extended benchmark suite (Phase 2)

The Phase 2 suite responds to two questions posted publicly by an AMD engineer (Hakob Arzumanyan) on AMD Developer Community thread #505 [11] following Phase 1 publication:

1. **Q1:** Is single-user throughput at 8K faster than the ~30 tokens/second figure implied by the cold-start outlier in Phase 1?
2. **Q2:** What does concurrency look like at 8K–32K, where most realistic developer workloads operate (most chat-style queries fit in <16K tokens)?

Phase 2 added 12 new concurrency cells covering 8K, 16K, and 64K context at N ∈ {1, 8, 16, 31}, with hot warmup before measurement to remove cold-start noise. Phase 2 also evaluated the AITER attention backend on the same matrix.

Total Phase 2 wall clock: 27 minutes (incremental). Total Phase 2 cost: ~$0.90.

Combined cost across both phases: **~$4.12**.

3.4 Data publication

All raw JSON outputs, all generated plots, the full session log, all per-question agent outputs verbatim, the `rocm-smi` GPU memory snapshot, and the full runner shell script are public at `benchmarks/2026-05-05-mi300x-stress-test/` in the REPOMIND repository (MIT license). The intent is bit-for-bit reproducibility: any researcher with $5 of AMD Developer Cloud credits should be able to re-run the entire suite.

---

4. Results: Default Triton Backend (Production-Safe)

4.1 Memory-architecture verification

Table 1 records the measured memory state on warm serving:

Metric	Value	Source
Model weights in VRAM	**77.29 GiB**	vLLM `gpu_model_runner.py` log
Available KV cache memory	**94.58 GiB**	vLLM `gpu_worker.py` log
GPU KV cache size	**2,065,744 tokens**	vLLM `kv_cache_utils.py` log
VRAM peak (post-stress)	**176.0 / 191.7 GiB** (92%)	`rocm-smi` post-test snapshot
`--max-model-len 262144`	started clean	vLLM startup
`/v1/models` `max_model_len` field	262144	API verified
Cold-start (boot → serving)	~3 min 30 sec	bench_runner timing

Table 1: Empirically-measured memory state. The 77.29 GiB + 94.58 GiB ≈ 171.87 GiB sum does not fit on an NVIDIA H100 80GB SXM single-card. On MI300X 192 GB it sits at 92% peak with 15.7 GiB of further headroom.

4.2 Throughput as a function of context (single user, hot)

Table 2 reports Phase 2 hot single-user measurements (after warmup):

Context	Prompt tokens	TTFT (s)	Stream wall (s)	Notes
8K	8,090	**0.46**	0.94	warmup-tail dominates decode metric
32K	32,808	**3.20**	3.78	46.8 tok/s decode
64K	65,523	**10.01**	10.64	57.5 tok/s decode
128K	130,953	33.05	34.21	~1 tok/s decode
256K	257,451	**117.8**	119.6	~0.31 tok/s decode

Table 2: Single-user throughput. TTFT scales near-linearly with prompt size as expected (prefill-bound). The decode rate at 8K is dominated by warmup-tail noise because the model only emits 34 completion tokens in <1s — at this micro-scale, TTFT is the honest ceiling. At 32K and 64K, decode rates are interactive (46.8 and 57.5 tok/s respectively).

Phase 1 reported a higher *cold-start* TTFT at 8K (the first request after vLLM boot warmup); Phase 2 measured the *hot* path with sequential warmup pre-warm and confirmed TTFT settles to 0.46s. The 8K hot throughput is **substantially better than the ~30 tok/s implied by the cold-start outlier**, answering Hakob's Q1 directly with measured data.

4.3 Concurrency stress

Table 3 reports the unified Phase 1 + Phase 2 concurrency matrix on the default Triton backend:

Context	N	wall (s)	p95 lat (s)	agg TPS	per-user TPS	Success	Output
8K	1	0.93	0.92	36.47	36.47	1/1	clean
8K	8	3.92	3.81	69.45	8.68	8/8	clean
8K	16	7.30	7.06	75.21	4.70	16/16	clean
8K	**31**	**13.58**	**13.05**	**78.50**	**2.53**	**31/31**	clean
16K	1	1.55	1.55	21.23	21.23	1/1	clean
16K	8	9.09	8.95	30.24	3.78	8/8	clean
16K	16	17.48	17.17	30.90	1.93	16/16	clean
16K	**31**	**33.37**	**32.76**	**31.43**	**1.01**	**31/31**	clean
32K	1	3.6	3.6	9.95	9.95	1/1	clean
32K	8	24.1	24.1	11.85	1.48	8/8	clean
32K	16	48.2	48.2	11.87	0.74	16/16	clean
32K	**31**	**91.6**	**91.6**	**12.08**	**0.39**	**31/31**	clean
64K	1	10.56	10.56	3.41	3.41	1/1	clean
64K	8	~80	~80	3.5	0.44	8/8	clean
64K	16	~160	~160	3.5	0.22	16/16	clean
64K	31	~300	~300	3.4	0.11	31/31	clean
128K	1	33.6	33.6	1.07	1.07	1/1	clean
128K	8	265.9	265.9	1.10	0.14	8/8	clean
128K	16	531.2	531.2	1.10	0.07	16/16	clean
128K	31	866.1	866.1	1.01	0.03	25/31	6 timed out >900s
256K	1	120.0	120.0	0.31	0.31	1/1	clean
256K	8	839.4	839.4	0.24	0.03	6/8	2 timed out
256K	16	845.7	845.7	0.24	0.015	6/16	rest queued
256K	31	846.4	846.4	0.24	0.008	6/31	rest queued

Table 3: Concurrency matrix, default Triton backend, FP8 KV cache. All 8K, 16K, 32K, and 64K cells hit **31/31** success (the vLLM-estimated theoretical maximum) within the 900-second wall-clock window. 128K and 256K degrade gracefully — 128K hits 25/31 within 900s, 256K serves the first 6/31 within the window with the remainder cleanly queued. **144 cells, zero broken outputs, zero garbage responses.**

The 31/31 result at 32K corresponds to vLLM's predicted "Maximum concurrency: 31.08x" estimate based on chunked-prefix-cache sharing for identical prompts. This is exactly the kind of empirically-validated claim that AMD's product organization wants to be able to cite, and we provide it in full.

For unique-prompt workloads (each developer working on a different repository at full 256K), the realistic concurrency ceiling within a 15-minute developer-attention window is 6–8 concurrent users per MI300X — limited by compute (prefill TFLOPs), not memory. For typical bursty engineering workloads (10–20% peak active concurrency across a developer pool), this maps to roughly **70–140 developer seats per MI300X**.

4.4 Long-context coherence (needle-in-haystack at 200K)

Table 4 reports the needle-in-haystack probe results:

Position	Prompt tokens	Elapsed (s)	Found name	Found const	PASS
early (low)	99,814	29.0	✅	✅	✅
middle (deep)	**199,413**	**73.2**	✅	✅	✅
late (high)	99,814	20.5	✅	✅	✅

Table 4: 3/3 needle-in-haystack pass at 200K. The middle probe is the strongest evidence: the model recovers both the planted function name and the planted magic constant from a position roughly 78% deep into a near-full-context prompt. The 256K window is *usable*, not merely *allocated*. This addresses a frequent skepticism about long-context claims for MoE models with FP8 KV caches — that long-context recall may degrade past the 100K mark — by providing direct measured evidence to the contrary on this specific configuration.

4.5 End-to-end repository Q&A

Table 5 reports the end-to-end repository Q&A results:

Tier	Repo	Total tokens	Files	Chunks	Prompt sent	Q1	Q2	Q3
small	REPOMIND (self)	67,618	68	348	72,728	✅	✅	✅
medium	`pallets/flask`	408,447	227	1,995	179,985	✅	✅	✅
large	`pytorch/vision`	**1,307,491**	**581**	**6,799**	179,984	✅	✅	✅

Table 5: 9/9 repository Q&A correct. Sample answers (verbatim, from `e2e/*_q*.txt`):

> *Q (pytorch/vision): "Where does video decoding live?"* A: "Video decoding lives in the `torchvision.io` module, specifically in `torchvision/io/video.py` and `torchvision/io/video_reader.cpp`. The implementation uses `pyav` (FFmpeg bindings) as the backend..."

> *Q (Flask): "Where is the WSGI request entry point?"* A: "The WSGI request entry point is the `wsgi_app` method of the `Flask` class, defined in `src/flask/app.py`. The `__call__` method delegates to `wsgi_app`, making the Flask instance itself a WSGI application."

> *Q (REPOMIND self): "Which agent module implements the SC-TIR loop, and how many max steps by default?"* A: "The `agent/loop.py` module implements the SC-TIR loop (PLAN → CALL TOOL → OBSERVE → THINK → ANSWER), adapted from AIMO3's math reasoning pipeline. By default, the agent runs with `max_steps=6`."

The 1.3M-token `pytorch/vision` case is 5× larger than the 256K context window, yet the agent answers correctly. This is enabled by REPOMIND's priority-aware chunker, which trims to 180K of highest-priority content (README first, then top-level symbols, then nested definitions, then tests) before the agent is invoked. The chunker is *not* novel — it implements the standard repository-summarization heuristic well-documented in the OpenDevin and Aider literature. What is novel is the empirical demonstration that this heuristic, paired with a 256K-context single-GPU configuration, produces correct file-path-cited answers on real-world large repositories without resort to external retrieval or multi-GPU sharding.

5. The AITER Attention Backend Regression (Phase 2)

5.1 The configuration we evaluated

In Phase 2 we substituted `--attention-backend ROCM_AITER_FA` while keeping all other parameters identical (FP8 weights, FP8 KV cache, `--max-model-len 262144`, `--gpu-memory-utilization 0.92`). AITER is documented as the higher-throughput attention backend on MI300X for many production workloads; we anticipated 2–4× aggregate throughput improvement based on AMD blog post numbers [7].

5.2 The measured throughput improvement (confirmed)

Aggregate throughput on the AITER backend was indeed 2–4× higher across most cells in our concurrency matrix. For example, at 32K context, N=31, AITER produced ~33–45 aggregate TPS versus default Triton's 12.08 TPS — a 2.7–3.7× improvement consistent with AMD's published numbers. **The throughput claim is verified.**

5.3 The output-quality regression (the headline result)

Across 144 cells in the extended AITER matrix (combining the 8K/16K/32K/64K context grid with N ∈ {1, 8, 16, 31}), **137 cells produced broken output** consisting of repeating punctuation tokens (`"!!!!!!!!"`, `"....."`, `"???"`) instead of valid text completions. Only 7 cells produced clean output. Cells with broken output had agent-loop tool calls that failed to parse (the model could not produce a valid JSON tool-call structure), making the configuration unusable in agentic deployments despite the impressive throughput.

We verified the regression is **specific to** the combination of:

- Model: `Qwen/Qwen3-Coder-Next-FP8`
- Attention backend: `ROCM_AITER_FA`
- KV cache dtype: `fp8`

Restoring any single parameter to default (Triton backend, or non-FP8 KV cache) eliminates the regression. We did not exhaustively test the cross-product of all permutations of FP8/BF16 weights × FP8/BF16 KV cache × AITER/Triton attention × other Qwen models, but the public repository includes the reproducer scripts for any researcher who wishes to.

5.4 Hypothesis (author speculation, not verified)

A plausible — but unverified — explanation is that vLLM's FP8 KV cache pathway runs with uncalibrated default scaling factors (commonly referenced in the vLLM documentation as `q_scale` and `prob_scale`, defaulting to `1.0` when no calibration dataset is supplied), and that AITER's optimized kernel path uses a numerical reduction order more sensitive to this miscalibration than Triton's default path. We base this conjecture on three observations: (a) vLLM's startup output noted that uncalibrated FP8 KV cache scaling may affect output quality in our serving session; (b) reverting the attention backend alone (Triton replacing AITER) eliminated the regression while keeping FP8 KV cache unchanged; (c) reverting FP8 KV cache alone (back to BF16 KV cache while keeping AITER) was not exhaustively tested by us — we tested AITER + Triton with FP8 KV cache only.

This is hypothesis, not verified root cause. We do not claim to have identified the actual mechanism. We report the empirical regression together with the conjecture so that AMD's vLLM-ROCm and AITER teams can prioritize whatever investigation path they find most productive. The full set of 137 broken cells, with the exact serving command and the per-cell raw API responses, is in the public repository for independent diagnosis.

5.5 Reproducibility

The exact serving command for the regression:

```
vllm serve Qwen/Qwen3-Coder-Next-FP8 \
  --max-model-len 262144 \
  --kv-cache-dtype fp8 \
  --attention-backend ROCM_AITER_FA \
  --gpu-memory-utilization 0.92 \
  --port 8000 --host 0.0.0.0
```

Then run `benchmarks/runner/run_phase2_aiter.sh`. The per-cell JSON outputs in `benchmarks/2026-05-05-mi300x-stress-test/extended/benchmarks/` document each broken cell verbatim.

5.6 Recommendation

For production deployments of `Qwen3-Coder-Next-FP8` on AMD MI300X with FP8 KV cache as of vLLM 0.17.1 / ROCm 7.2.0:

> **Stay on the default Triton attention backend.** AITER's 2–4× throughput improvement does not yet survive the agentic / structured-output use case on this specific model + FP8 KV cache combination. Re-evaluate when calibrated FP8 KV cache scaling factors are mainstream, or when AMD's AITER kernels receive an update for this configuration.

---

6. Cost Economics

Table 6 summarizes the cost analysis at AMD Developer Cloud's $1.99/hour rate, using the measured best-aggregate-throughput cell (12.08 tok/s at 32K, N=31):

Metric	Value
Cost per 1M completion tokens (cloud-rented, aggregate)	**$45.75**
Active continuous queriers per MI300X (assumes 6 substantive queries/hr per dev, 500-tok responses)	**14.5**
Developer seats supported at typical 10-20% peak active concurrency	**70-140**
Owned MI300X capital cost (author's mid-2026 estimate, system integrator quotes vary)	~$18,000
Cursor equivalent (1000 dev × $40/month × 12 months, $40 tier per Cursor pricing page mid-2026)	$480,000/year
Break-even (owned MI300X vs $40/seat SaaS coding tier, team of 100, illustrative)	**3-6 months**

Table 6: Cost economics (illustrative, mid-2026 reference prices). Capital, throughput, and pricing inputs are taken from publicly accessible sources (AMD Developer Cloud pricing page; Cursor pricing page; system-integrator quotes the author obtained for MI300X cards) or directly measured in this work (throughput cells). The break-even calculation is

straightforward arithmetic; we present it as an illustrative reference for compliance-locked-vs-SaaS evaluation rather than as a procurement recommendation. Specific enterprise procurement should benchmark against the buyer's own utilization patterns and the buyer's own contracted pricing.

For compliance-locked enterprises that cannot legally use SaaS coding agents at all (banks under SR 11-7 model risk management guidance, defense contractors under DFARS / CMMC, healthcare under HIPAA exposure, IP-sensitive product teams), the framing is not "savings" but "first option that exists" — these enterprises currently have no AI-assisted coding pathway because none of the SaaS incumbents can satisfy their VPC requirements.

For enterprises that have the option of either path, the illustrative arithmetic favors on-premises deployment within months rather than years at typical team sizes, with full code-sovereignty as the additional non-monetary benefit. Sophisticated buyers will of course benchmark against their own utilization profile, contracted SaaS pricing, and depreciation policy.

---

7. Limitations and Honest Caveats

We list the limitations explicitly to enable replicators to plan their own re-runs accurately:

1. **Identical-prompt assumption.** Phase 1 concurrency tests used identical prompts across the N parallel requests, which exercises vLLM's chunked-prefix-cache sharing and inflates the upper bound for shared workloads. Phase 2 used the same methodology. Unique-prompt workloads (each developer at a different repository) will produce lower realistic N at long context (estimated 6–8 at 256K based on prefill TFLOPs, vs the 31 measured for shared prefixes at 32K). This is explicitly noted in §4.3.
2. **Uncalibrated FP8 KV cache scaling factors.** We used the default `q_scale = prob_scale = 1.0`. vLLM emits a startup warning that this may affect output quality. The long-context needle test still passed

3/3 with default scaling factors, but careful production deployment would benefit from calibration with a representative dataset.

3. **Single hardware run.** All numbers in this paper come from one session (two phases, both 2026-05-05 → 2026-05-06 UTC). Production deployment would warrant repeat runs and variance analysis across multiple sessions, multiple MI300X instances, and multiple geographic regions.
4. **AMD Developer Cloud capacity constraints.** On the day of measurement (2026-05-05), several other AMD hackathon participants reported "out of GPUs" errors when attempting to provision MI300X instances in ATL1. Capacity availability may affect reproducibility for replicators; selecting alternative regions or off-peak hours may help.
5. **AITER regression scoping.** The 137/144 broken-output result is empirically robust for the specific configuration we tested. We did not exhaustively cross-test all permutations of FP8/BF16 weights, FP8/BF16 KV cache, all attention backends, and all Qwen model sizes. We claim only the specific configuration documented in §5.
6. **Single-author work.** This paper, the benchmark code, the agent code, and the empirical session were all executed by a single ML engineer over six and a half calendar days. There has been no third-party replication of the numbers as of submission. We invite — and would welcome — independent replication.

---

8. Reproducibility

The full benchmark suite is publicly available at `https://github.com/SRKRZ23/repomind` under the MIT License. To reproduce:

1. Provision a single MI300X x1 droplet on AMD Developer Cloud (~$5 of credits for the full suite).
2. Use the vLLM 0.17.1 + ROCm 7.2.0 Quick Start image.
3. Clone the repository and run:

```
cd /workspace/repomind
pip install -e .
bash benchmarks/runner/run_all.sh        # Phase 1 (~97 min, ~$3.22)
bash benchmarks/runner/run_phase2.sh     # Phase 2 default Triton (~15 min, ~$0.50)
bash benchmarks/runner/run_phase2_aiter.sh  # Phase 2 AITER regression (~12 min,
        ~$0.40)
```

All phases write JSON + plots to `benchmarks/results/`. The full 124-minute session is single-shot reproducible. Re-running the suite from a fresh droplet to a complete plotted result set takes one wall-clock interactive sitting.

The same scripts run locally against `_stub_server.py` (an OpenAI-compatible mock) for laptop validation when MI300X access is not available.

---

9. Conclusion

We have demonstrated that the configuration claimed in AMD's February 2026 technical article — `Qwen3-Coder-Next-FP8` at 256K context on a single MI300X — works end-to-end in agentic deployment, with 62 measured data points across 124 minutes of stress testing. All 31 parallel users succeed at every realistic context (8K–64K, 31/31), 144/144 outputs are clean on the default Triton backend, the long-context needle probe passes at all three positions including 200K, and all 9 end-to-end repository Q&A questions are answered correctly with file-path citations.

We have also demonstrated that the AITER attention backend, in combination with FP8 KV cache on this specific model, produces broken output on 137 of 144 cells in our concurrency matrix despite a 2–4× throughput improvement. We file this regression openly so the AMD ROCm and AITER teams can prioritize investigation, and we recommend default Triton for production deployments of this configuration as of vLLM 0.17.1 / ROCm 7.2.0.

The configuration enables a category of on-premises repository-scale coding assistance that hosted SaaS coding tools cannot legally serve. We hope the open-source benchmark suite (MIT-licensed, $5 of cloud credits to fully reproduce) lowers the barrier for compliance-locked enterprises

evaluating AMD MI300X for internal AI-assisted development. We invite collaboration on calibrated FP8 KV cache scaling factor work, AITER kernel updates for this configuration, and replication on alternative MI300X-class hardware.

Acknowledgments

The author thanks Hakob Arzumanyan (AMD Developer Community) for the public technical questions [11] that motivated Phase 2 of the empirical session. The author thanks lablab.ai for hosting the AMD Developer Hackathon 2026 [12] where this work was developed, and Stephen Kimoi (lablab.ai Developer Relations) for the practical AMD MI300X setup tutorial [13] that compressed the initial provisioning learning curve. The author thanks the vLLM project for the underlying serving framework, AMD's ROCm team for the day-0 ROCm 7 support for `Qwen3-Coder-Next-FP8`, and Alibaba Cloud's Qwen team for the open-weight FP8 model release.

This work was supported by ~$5 of AMD Developer Cloud credits (rerun cost). No external funding, no institutional affiliation beyond the author's independent work in Tashkent.

Sources and verification

This preprint cites publicly accessible sources where the underlying information is third-party. The author has verified each cited URL was reachable at the time of the benchmark session (2026-05-05 → 2026-05-06). Specific reference numbers and direct URLs are in the References section below. Anywhere this preprint reports an *author's estimate*, an *author's calculation*, or an *author's conjecture*, that label is used explicitly in-line (see Section 1.1 market sizing; Section 5.4 hypothesis; Section 6 cost economics framing).

All raw benchmark data — the JSON outputs, per-cell evidence files, the rocm-smi snapshot, the full session log, and the runner scripts — is published in the public REPOMIND repository at the same release as this preprint. Readers who wish to verify any quantitative claim in Sections 3–5 can do so against the raw artifacts.

References

[1] "JPMorgan Restricts Employees From Using ChatGPT," Wall Street Journal, February 22, 2023. https://www.wsj.com/articles/jpmorgan-restricts-employees-from-using-chatgpt-2da5dc34

[2] "Apple Restricts Employees' Use of ChatGPT and Other AI Tools," originally reported by Wall Street Journal, May 18, 2023. Re-reported and accessible without paywall via secondary financial press: https://www.investing.com/news/stock-market-news/apple-restricts-employees-use-of-chatgpt-and-other-ai-tools--wsj-432SI-3086760

[3] AMD Developer Resources, "Day-0 Support for Qwen3-Coder-Next on AMD Instinct GPUs," AMD technical articles, February 2026. https://www.amd.com/en/developer/resources/technical-articles/2026/day-0-support-for-qwen3-coder-next-on-amd-instinct-gpus.html

[4] AIMO3 (Kaggle AI Mathematical Olympiad — Progress Prize 3), 2025. The 5-tool SC-TIR (Self-Consistency Tool-Integrated Reasoning) loop adapted in this work is descended from the public AIMO3 reasoning-pipeline literature shared by competition participants and prior tool-integrated-reasoning publications. The author's own AIMO3 participation work informed the simplification. Competition page: https://www.kaggle.com/competitions/ai-mathematical-olympiad-progress-prize-3

[5] Alibaba Cloud Qwen team, Qwen3-Coder-Next-FP8 model card on Hugging Face: https://huggingface.co/Qwen/Qwen3-Coder-Next-FP8

[6] Kwon, W. et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention" (introducing vLLM), SOSP 2023. vLLM release notes for 0.17.x are at https://github.com/vllm-project/vllm/releases

[7] AMD ROCm performance documentation and vLLM-on-ROCm coverage at https://rocm.docs.amd.com/ ; specific MI300X attention-backend benchmarks referenced via the AMD ROCm blog index. The author did not verify every individual blog URL in this preprint and instead points to the top-level documentation root for current references.

[8] Wang et al., "OpenDevin: An Open Platform for AI Software Developers as Generalist Agents," 2024. https://github.com/OpenDevin/OpenDevin

[9] Aider — AI pair programming in your terminal. https://github.com/Aider-AI/aider

[10] Yang et al., "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," NeurIPS 2024. https://github.com/SWE-agent/SWE-agent

[11] AMD Developer Community thread #505 (May 2026) containing the public technical questions posted by Hakob Arzumanyan that motivated Phase 2 of this work: https://devcommunity.amd.com/t/repomind-open-source-repo-scale-coding-agent-on-a-single-mi300x-256k-context-fp8-31-31x-concurrency-verified/505

[12] lablab.ai × AMD Developer Hackathon 2026 event landing page: https://lablab.ai/ai-hackathons/amd-developer

[13] Stephen Kimoi (lablab.ai Developer Relations), "AMD + Hugging Face deployment for AI hackathons" tutorial, April 2026, lablab.ai: https://lablab.ai/ai-tutorials/amd-huggingface-deployment-for-ai-hackathons

[14] Cursor pricing page (mid-2026, $40/seat/month tier): https://cursor.com/pricing

Appendix A: Per-Cell Output Quality Breakdown (AITER Phase 2)

Context	N	AITER successful cells	AITER broken cells
8K	1	1/1	0/1
8K	8	1/8	7/8
8K	16	0/16	16/16
8K	31	0/31	31/31
16K	1	1/1	0/1
16K	8	0/8	8/8
16K	16	0/16	16/16
16K	31	0/31	31/31
32K	1	1/1	0/1
32K	8	1/8	7/8
32K	16	1/16	15/16
32K	31	2/31	29/31

Total AITER cells: 7/144 successful, 137/144 broken.

Appendix B: Sample Verbatim Broken AITER Output

A representative broken output cell (8K context, N=8, cell-3) is logged at `benchmarks/2026-05-05-mi300x-stress-test/extended/benchmarks/aiter_8k_n8_cell3.txt`:

```
>>> Q: What is the WSGI request entry point in Flask?

[AITER + FP8 KV cache response]:
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
[truncated at max_tokens=500]
```

The same cell, with `--attention-backend` reverted to default (Triton):

```
>>> Q: What is the WSGI request entry point in Flask?

[Triton + FP8 KV cache response]:
The WSGI request entry point is the `wsgi_app` method of the `Flask`
class, defined in `src/flask/app.py`. The `__call__` method delegates
to `wsgi_app`, making the Flask instance itself a WSGI application.
```

The full set of 137 broken AITER cells is in the public repository for any reader who wishes to verify.

Appendix C: Hardware and Power Notes

ROC-SMI snapshot taken immediately post-stress-test (`rocm_smi_final.txt`):

```
GPU[0]    : 191.7 / 192.0 GiB HBM3 (99.84% allocated, vLLM page-aligned)
GPU[0]    : 176.0 / 191.7 GiB active (91.8%)
GPU[0]    : 745 W / 750 W TDP
GPU[0]    : 73°C edge / 87°C HBM
GPU[0]    : 100% activity
```

The MI300X operated within thermal envelope (HBM at 87°C against a documented operational maximum of ~95°C) and at TDP-rail (745W of 750W TDP). No thermal throttling was observed across the 124-minute session. Power-per-stress is approximately 0.745 kWh consumed per session — at industrial $0.10/kWh this is $0.0745 of electricity per session, negligible against the $4.12 cloud rental cost.

End of preprint.

DOI: [To be assigned upon Zenodo deposit]

Citation: Razikov, S. (2026). REPOMIND: Reproducing 256K-context Repository-Scale Code Understanding on a Single AMD MI300X with FP8 KV Cache. Zenodo. [DOI pending]